



QIGen: A Kernel Generator for Inference on Nonuniformly Quantized Large Language Models

Tommaso Pegolotti
ETH Zurich
Zürich, Switzerland
tommaso.pegolotti@inf.ethz.ch

Dan Alistarh
ISTA
Vienna, Austria
dan.alistarh@ist.ac.at

Markus Püschel
ETH Zurich
Zürich, Switzerland
pueschel@inf.ethz.ch

Abstract—Efficient inference on large language models (LLMs) has become a popular topic in both academia and industry. Roughly speaking, LLMs consist of a collection of weight matrices, and generative inference on these models essentially computes a sequence of matrix-vector products and thus can be heavily memory-bound. Consequently, much work has been devoted to reducing the size of the weights to lower bit-widths through various forms of quantization. In turn, these diverse precision formats complicate both the arithmetic and optimized kernel implementation. So far, the vast majority of implementation work for mixed-precision LLM computation has been done manually. Currently, one of the most powerful and complex scalar LLM compression techniques is *nonuniform* quantization, in which a matrix is divided unevenly into parts that are quantized with different bit-widths, minimizing the output compression error. In this paper, we present QIGen, the first kernel generator for LLM inference on CPUs to support nonuniform quantization in full generality. Given a nonuniformly quantized LLM and target CPU characteristics, QIGen first generates the diverse set of needed custom matrix-vector product kernels and combines them with a suitable storage format. We benchmark and analyze QIGen-generated code in various experiments. In particular, we show that our code achieves Pareto optimality in terms of performance and accuracy with respect to the most used open-source tool. We show a speedup of up to $1.3\times$ for the matrix-vector and $3.4\times$ for the matrix-matrix computations even when using uniform quantization.

Index Terms—Neural network inference, program generation, performance, quantization, compression, SIMD vector code

I. INTRODUCTION

The impressive performance of generative large language models (LLMs) (1; 2; 3) has led to significant interest in executing them on user devices with limited computational power, including CPUs. Interestingly, the computational structure of generative LLM workloads renders this plausible. LLMs generate one token (word) at a time based on a cached context from the previously generated word, and the computational cost of each such step is relatively small. Concretely, each step consists of a sequence of memory-bound matrix-vector products instead of computationally intensive tensor computations or convolutional operations that are otherwise common in deep learning.

Model quantization. Thus, the critical bottleneck in generative inference on commodity devices such as CPUs becomes *memory movement*: well-performing models have parameter counts in the billions or tens of billions (e.g. (2; 3)), which

often exceed the memory capacity of consumer devices and induce high memory transfer costs at runtime, overwhelming bandwidth. To address this issue, a series of accurate *quantization-based compression methods* specialized to LLMs have recently been proposed (4; 5; 6; 7; 8; 9; 10). Such methods work by reducing the bit-width of data types in the model used for storing matrix weights, activations, or both, to minimize the impact on accuracy. State-of-the-art methods do so in customized packed numerical formats, which work by jointly quantizing contiguous groups of weights using the same scale and zero-point. Existing methods can quantize LLMs accurately to approximately 3–4 bits per parameter (6; 10), down from the standard half-precision (FP16).

System support for quantized models. Given this algorithmic progress, a remaining challenge is *efficient system support* for these compressed numerical formats, which would allow LLMs to execute on end devices. For GPUs, this was initiated by academic proposals such as LLM.int8() (4), SmoothQuant (8), GPTQ (6), or AWQ (10), and has been integrated into professional-grade tools such as vLLM (11) or TensorRT-LLM (12). At the same time, local inference on CPUs is supported via extremely popular open-source local inference solutions such as llama.cpp (13) or MLC (14), with hundreds or even thousands of contributors. However, due to programming complexity and hardware diversity, there is still a lack of robust support for the entire range of available *nonuniform quantization* techniques, which have been shown to provide better accuracy per average bit-width (e.g., (15)) by varying the quantization granularity across different parts of the same weight matrix. Specifically, a major challenge is the large diversity of computational kernels required by the nonuniformity, which may explain why to date it is not supported by any of the systems above.

Currently, the community develops custom kernels *manually* for each compressed format and each target architecture. Unfortunately, this approach can be extremely time-intensive, error-prone, and non-portable, as it requires potentially re-writing kernels from scratch to support new quantization formats and target hardware.

Contribution. In this paper, we present QIGen, the first kernel generator for efficient inference with quantized LLMs on CPUs, which also supports nonuniform quantization in full generality. QIGen generates highly optimized code for

the diverse set of matrix-vector and matrix-matrix product kernels needed to support LLM inference with nonuniformly quantized weight matrices.

Specifically, we offer the following contributions:

- We present the design and implementation of QIGen, a kernel generator for efficient quantized inference with LLMs on CPUs. QIGen takes as input quantized weight matrices of an LLM, which can be produced using popular open-source tools (6). Based on the characteristics of the matrices, QIGen generates highly optimized and vectorized LLVM-IR code (16) for the matrix-vector and matrix-matrix products needed during inference. These functions are then just-in-time compiled and assigned to each layer of the LLM.
- In particular, QIGen supports *nonuniform* quantization, which uses different quantization parameters for different parts of an LLM weight matrix and thus requires the generation of a diverse set of kernels with different bit-widths and storage formats.
- We provide a bit packing strategy that can support the full range of nonuniform quantization methods and bit-widths and is fully vectorizable.
- We provide a C++ implementation of QIGen. All code is available at <https://github.com/tpgolotti/QIGen>.
- We illustrate how code generation simplifies the process of retargeting our implementation to different ISAs.
- We show experimental results on the accurate LLaMA3 family (3) of LLMs. In particular, we examine the benefits of nonuniform quantization in terms of the trade-off between accuracy and memory/performance. Additionally, we compare the runtime of our generated kernels to llama.cpp, an extremely popular open-source tool (13).

II. BACKGROUND

We provide background on LLM generative inference. At a high level, the task of generating the next token (sub-word), given a cached context (pre-amble), can be mapped to a series of matrix-vector (or sometimes matrix-matrix) multiplications, where each matrix corresponds to a layer's weights, and the vector corresponds to the embedding of the word being generated. The critical challenge is the large size of the weight matrices (e.g., $14K \times 4K$ elements), which puts significant pressure on the CPU memory hierarchy. These multiplications constitute the kernels of LLM inference, determine its performance, and are the target of our proposed kernel generator QIGen.

We then review the nonuniform quantization technique that we leverage to speed up LLM inference by compressing the weight matrices. This paper's key contribution is that QIGen supports nonuniform quantization in full generality.

LLM inference. Computationally, LLM generative inference is a sequence of matrix-matrix (GEMM) and vector-matrix (GEVM, matrix from the right) multiplications by the weight matrices W in a neural network, one for each layer.

In the most common case, the network is given an input text with n words. Each word is converted into a vector using

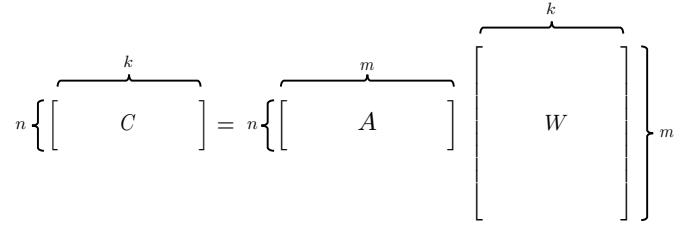


Fig. 1: High-level visualization of the kernels needed in LLM inference and targeted by QIGen. The input matrix A is of size $n \times m$, the weight matrix W is of size $m \times k$, and the output C is of size $n \times k$. n is small and most important is $n = 1$, while m and k are large. In BLIS, these are called GEPM.

```

j ← 0
while j < k do
  Wb ← pack(W[:, j:j+kb])
  Cb ← pack(C[:, j:j+kb])
  Cb = AWb
  C[:, j:j+kb] ← unpack(Cb)
  j ← j + kb
end while

```

Fig. 2: High-level implementation of a GEPM function.

an embedding function to yield an $n \times m$ matrix A . First, in the *prefill phase*, A is run through the network, each layer computing a GEMM. Finally, this phase produces one token (sub-word) as output. In the *generation phase*, the generated token is again converted into a vector (now $n = 1$), fed through the network, each layer now computing GEVMs to produce the next token, and so on. Thus, for moderate-sized output sequences, most multiplications during a prefill and generation sequence are GEVMs and constitute the bottleneck. Some techniques generate more than one word per inference(17; 18), usually n between 1 and 8, which we also support with a specialized GEMM instead of GEVM. The basic computations needed are shown in Fig. 1. To structure the needed GEMMs and GEVMs, we use ideas from the BLIS framework (19; 20), in which computations of the form in Fig. 1 (i.e., n is small) are called GEPM (panel matrix multiplication). We compute them as depicted in Fig. 2, using `numpy` array notation. Namely, the matrices C and W are split along the k dimension into blocks C_b and W_b of size $n \times k_b$ and $m \times k_b$ respectively, which are stored sequentially in memory with a call to `pack`. Then, each C_b is computed as AW_b and stored back into C . In practice, to avoid allocating nonuniform memory for each GEPM execution, we only pack W at compile time and use software prefetching for C . For larger n , we compute the resulting GEMMs as a sequence of GEPMs by blocking in the n dimension.

As explained, LLM inference operates with inputs with small n , and mostly $n = 1$. Therefore, the computation is equal or close to a GEVM and thus memory-bound. Thus, an avenue for higher performance is the compression of the

LLM's weight matrices W to contain smaller data types. This compression has to be done carefully to maintain the model's performance. An approach that has proven successful is a suitable form of quantization (6), which, however, comes with the challenge of computing with non-common data types.

Quantization. The general goal of quantization is to lossily compress real numbers (or, when implemented, floating-point numbers) by mapping them to a limited set of integer levels. We consider in this work two quantization schemes, one for the weight matrix and one for the input.

Asymmetric quantization. For the weight matrices W we use an asymmetric quantization scheme. These use a uniform affine quantization function, meaning that given a vector $\mathbf{x} \in \mathbb{R}^n$, we define

$$\begin{aligned} Q(\mathbf{x}, b)_a &= \text{rnd} \left(\frac{\mathbf{x} - \min(\mathbf{x})}{\max(\mathbf{x}) - \min(\mathbf{x})} (2^b - 1) \right) \\ &= \text{rnd}((\mathbf{x} - z)/s(\mathbf{x})), \end{aligned} \quad (1)$$

where rnd rounds to nearest, $z = z(\mathbf{x}) = \min(\mathbf{x})$ maps to zero and $s = s(\mathbf{x}) = (\max(\mathbf{x}) - \min(\mathbf{x})) / (2^b - 1)$ is the scale. For example, given a vector

$$\mathbf{x} = (3.8 \quad 2.4 \quad 3.0 \quad 4.2 \quad 3.9 \quad 1.0 \quad -1.0 \quad -2.2) \quad (2)$$

in $\mathbb{R}^8$, we have $s = 6.4 / (2^b - 1)$, $z = -2.2$. For $b = 2$ we would obtain

$$Q(\mathbf{x}, 2)_a = \mathbf{y} = (3 \quad 2 \quad 2 \quad 3 \quad 3 \quad 2 \quad 1 \quad 0). \quad (3)$$

Both z and s are stored with $\mathbf{y}$ to enable the (approximate) reconstruction

$$\tilde{\mathbf{x}} = D(\mathbf{y}, s, z) = s\mathbf{y} + z. \quad (4)$$

In the example

$$\tilde{\mathbf{x}} = (4.2 \quad 2.1 \quad 2.1 \quad 4.2 \quad 4.2 \quad 2.1 \quad -0.1 \quad -2.2). \quad (5)$$

The reconstruction error is computed as $\epsilon_r = \|\mathbf{x} - \tilde{\mathbf{x}}\|_2$. In (5), compared to (2), $\epsilon_r = 1.80$.

We can improve ϵ_r by partitioning $\mathbf{x}$ into groups and quantizing each group separately, thus storing s and z values for each group. For example, we can consider the left and right halves $\mathbf{x}_{0:3}$ and $\mathbf{x}_{4:7}$ in $\mathbf{x}$ in (2) and get

$$\mathbf{s} = (0.6 \quad 2.0) \quad \text{and} \quad \mathbf{z} = (2.4 \quad -2.2).$$

Quantizing again with $b = 2$, but now both halves separately, yields

$$(2 \quad 0 \quad 1 \quad 3 \quad 3 \quad 2 \quad 1 \quad 0) \quad (6)$$

with reconstruction

$$(3.6 \quad 2.4 \quad 3.0 \quad 4.2 \quad 3.9 \quad 1.9 \quad -0.2 \quad -2.2).$$

The reconstruction error reduces to $\epsilon_r = 1.21$ at the cost of storing two additional floating-point values for s and z . Intuitively, these two values better adapt the quantization range to the set of vector values.

Symmetric quantization. For the input matrices/vectors A we use a symmetric quantization scheme to take advantage

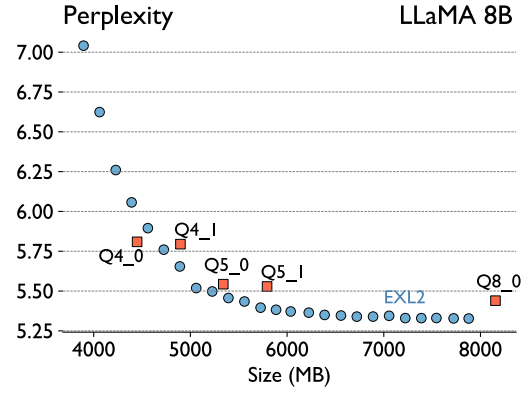


Fig. 3: Perplexity vs. size of LLaMA3 8B quantized using uniform (llama.cpp, red dots) and nonuniform (EXL2, light blue) formats. Perplexity is measured on WikiText2 with a stride of 512. Lower is better on both axes.

of the 8-bit integer operations provided by modern CPUs. It is a simplified implementation of the asymmetric quantization scheme that uses no zero-point value and is done on the fly. Given our example vector (2) we get with $b = 8$ the scale value $s = 6.4/127$ and the quantized vector

$$Q(\mathbf{x}, 8)_s = \mathbf{v} = (115 \quad 73 \quad 91 \quad 127 \quad 118 \quad 30 \quad -30 \quad -67),$$

similarly, we reconstruct the approximate value as

$$D(\mathbf{v}, s, 0) = s\mathbf{v}.$$

The techniques described in the following paragraph apply to both asymmetric and symmetric quantization schemes.

Nonuniform quantization. Most LLM quantization schemes are uniform, meaning that, across rows, all groups are of the same size and use the same bit-width (6; 10; 8). This simplifies the code, as the representation is constant, but it significantly restricts the opportunities for reduced storage and better performance. In contrast, *nonuniform quantization* varies the size and bit-width for each group; thus, one can better fit the matrix weight distribution by adapting the grouping to the range of elements in the matrix columns.

For LLMs, nonuniform quantization has been shown to lead to better accuracy (lower perplexity) at the same average bit-width and is supported by the popular EXL2 compressed format (15). We plot in Fig. 3 the accuracy, measured in model perplexity, versus the size of LLaMA3 8B quantized using a uniform quantization method (llama.cpp) and a nonuniform quantization format (EXL2). The former takes as input a single format type that is used for the whole model, the latter needs a target bit per weight value. We see that nonuniform quantization yields generally better Pareto-optimal trade-offs across the entire range of perplexities and memory requirements. As additional motivation, we plot in Fig. 4 the MMLU (Massive Multitask Language Understanding) evaluation of EXL2 compressed LLaMA3 8B and 70B models, which evaluate the ability to answer to more complex questions. Supporting nonuniform quantization allows for greater granularity

TABLE I: An example vector (first row) quantized in different ways, each with its own trade-off between reconstruction error and storage requirement. The first column also contains the equation number where it appears in the text.

variant	values	scale s	zero-point z	ϵ_r	size
initial vector (2)	(3.8 2.4 3.0 4.2 3.9 1.0 -1.0 -2.2)			0	32B
asymmetric quantized no split (3)	(3 2 2 3 3 2 1 0)	2.1	-2.2	1.80	10B
asymmetric quantized 4, 4-split (6)	(2 0 1 3 3 2 1 0)	(0.6 2.0)	(2.4 -2.2)	1.21	18B
nonuniform 6, 2-split (7)	(3 1 2 3 3 0 1 0)	(1.1 1.2)	(1.0 -2.2)	0.62	17.75B
nonuniform permuted 6, 2-split	(3 2 3 3 1 0 1 0)	(2.1 1.4)	(-2.2 2.4)	0.57	17.75B

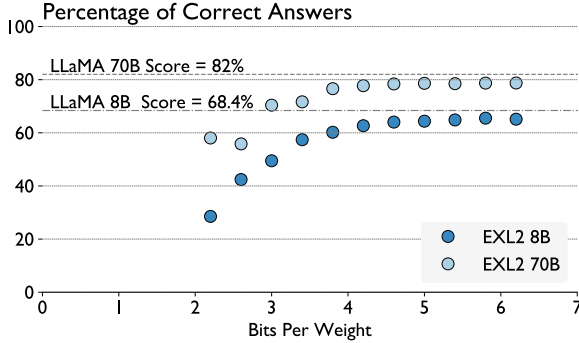


Fig. 4: MMLU evaluation of nonuniform (EXL2) quantized LLaMA3 8B (red) and 70B (blue) models to various target bits per weight values. (Higher is better.) Observe significant increases in model accuracy as we increase bit-widths to around 4 bits per parameter.

in selecting the optimal bit-widths for different parts of the model, which in turn allows better adaptation to the underlying hardware.

For example, consider the vector $\mathbf{x}$ in (2), where we quantize $x_{0:5}$ using 2 bits and $x_{6:7}$ using 1 bit, resulting in:

$$(3 \ 1 \ 2 \ 3 \ 3 \ 0 \ 1 \ 0). \quad (7)$$

This reduces the error to $\epsilon_r = 0.62$ with slightly less storage compared to the uniform 4,4-bit split in (6).

An additional technique to improve the reconstruction error is to first permute the vector before quantization, with the goal of grouping weights with a similar distribution, thus incurring a lower quantization error. For example, using a permutation π to swap the entries 2 and 6 in $\mathbf{x}$ and then quantizing with the same 6,2 split as before further reduces the reconstruction error to $\epsilon_r = 0.57$ (last row in Table I). The table shows that the trade-offs and best solutions are not obvious.

We provide an end-to-end kernel generator for inference in LLMs using nonuniformly quantized weight matrices W with permutations. The quantization is done via the popular open-source `ExLlamaV2`¹ (EXL2) package. This package first computes a suitable permutation of the rows in W and then quantizes each column in W nonuniformly, but using in each column the same partition and choice of bit-widths for

the groups. Since only one permutation is used, the cost of storing it is amortized. We denote the total number of groups used for W by $G = G_W$.

Supporting nonuniformity requires generating a diverse set of kernels to handle different group sizes and bit-widths. The key building block is the quantized dot product, which we discuss next.

Quantized dot product. Quantized inference requires multiplying the quantized weight matrix W with a floating-point vector, which, in turn, consists of quantized dot products. We continue the running example by considering again the vector $\mathbf{x}$ in (2), nonuniformly quantized with permutation to $\mathbf{y}$ as before, and the input vector of real values

$$\mathbf{t} = (1.5 \ 0.2 \ 0.3 \ 0.9 \ 2.6 \ 0.7 \ 0.9 \ 0.7) \in \mathbb{R}^8.$$

On entry to our kernels we symmetrically quantize the permuted input $\pi(\mathbf{t})$ to 8-bit as

$$\mathbf{v} = (73 \ 44 \ 15 \ 44 \ 127 \ 34 \mid 36 \ 127),$$

with scale vector $(0.02, 0.005)$. The quantized dot product is then computed as

$$\begin{aligned} & \langle D(\pi(\mathbf{v}), (0.02, 0.005), 0), D(\mathbf{y}, \mathbf{s}, \mathbf{z}) \rangle \\ &= \langle 0.02\mathbf{v}_{0:5}, 2.1\mathbf{y}_{0:5} - 2.2 \rangle + \langle 0.005\mathbf{v}_{6:7}, 1.4\mathbf{y}_{6:7} + 2.4 \rangle \\ &= 0.04\langle \mathbf{v}_{0:5}, \mathbf{y}_{0:5} \rangle - 0.04\langle \mathbf{v}_{0:5}, \mathbf{1} \rangle + \\ & \quad 0.007\langle \mathbf{v}_{6:7}, \mathbf{y}_{6:7} \rangle + 0.012\langle \mathbf{v}_{6:7}, \mathbf{1} \rangle. \end{aligned}$$

This operation consists of two main components, an integer dot product and a prefix sum of the input. In the following sections we explain how our kernels compute these operations.

Integer dot product. Modern x86 and ARM processors provide instructions for executing a dot product on 8-bit integers, such as `vpdpbusds` and `vdotq_u32`. In particular, the dot product is executed between groups of four 8-bit integers and the result is accumulated into 32-bit values. Since these instructions are made to work on 8-bit inputs, they are not directly practical for arbitrary low-bit inputs. However, since the weight matrices are known at compile time, we can store them efficiently to make use of these instructions as explained later in Section IV.

III. QIGEN: OVERVIEW

We provide an overview of QIGen, our proposed kernel generator for inference in nonuniformly quantized LLMs.

¹<https://github.com/turboderp/exllamav2>

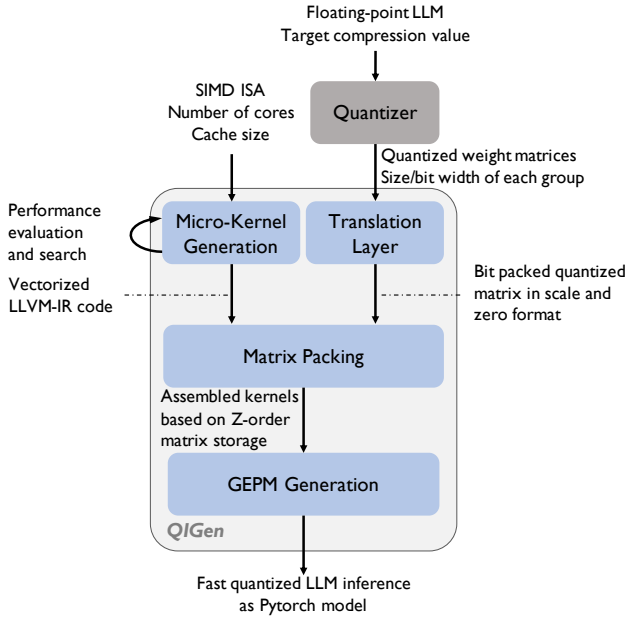


Fig. 5: High-level overview of the kernel generation pipeline in QIGen. The inputs are CPU specifications and a quantized LLM produced by an existing quantizer. The output is a highly optimized SIMD vector code for the GEPMs needed for quantized inference on this LLM, which is provided as a PyTorch model for ease of use. The internal steps are explained in the text.

A detailed description is then provided in the next section. QIGen’s high-level structure is shown in Fig. 5. It takes as input a nonuniformly quantized LLM and the characteristics of the target CPU, such as the instruction set architecture (ISA), number of cores, cache size, and vector register size. The output is highly optimized and vectorized LLVM-IR code of the needed GEPM kernels, which are then just-in-time compiled. These functions can be run from both C++ and Python.

Quantizer. The input to our program generator is a nonuniformly quantized model (see Section II), which is produced by an existing quantizer. This quantizer takes as input a floating-point LLM and a target compression rate, expressed in bits per weight. We need the output LLM of a quantizer to contain for each weight matrix W the number of groups G , their (varying) group sizes and their corresponding bit-width, written as (g_i, b_i) , $i = 1, \dots, G$. In this work, we use ExLlamaV2 as quantizer.

Translation layer. For fast implementation, our program generator uses an efficient way and particular format to store the quantized weights and the scale and zero values (s and z) needed to quantize and dequantize floating-point values via (1) and (4). For example, we store multiple low-bit integer weights into a single integer. Therefore, to make our code independent from the output format of the quantizer, we implement a translation layer that converts to our format. Additionally,

our generated code can operate on any bit-width, so we also implement a fast general packing algorithm to compress and decompress vectors. This translation needs to be done only once per model.

Micro-kernel generation. Since we operate on nonuniformly quantized matrices, we must support multiple combinations of group sizes and bit-widths for one quantized LLM. Therefore, we extract all the pairs (g, b) and generate optimized custom micro-kernels for each. The generation includes a form of autotuning by using a small model to determine the block size and searching over a small set of other relevant implementation choices. In practice, there are hundreds of possible choices for the needed kernels assuming bit values in 2, 3, 4, 5, 6, 8 and group sizes in 32, 64, 128. Autotuning can be run with various granularities. In the most expensive case, it can take up to days for larger models.

Matrix packing. Once the micro-kernels are generated, we iterate over the input model’s matrices and pack the weight values in microblocks using a particular order for best spatial locality and with the best block sizes found previously. We then store the matrices statically.

GEPM generation. Finally, we combine the function calls to different micro-kernels for each weight matrix in the quantized model to obtain the overall needed GEPM kernel functions. These functions are then assigned to each layer of the original PyTorch model, which can be run directly in Python.

IV. QIGEN: DETAILED DESCRIPTION

We now explain QIGen in detail, as depicted in Fig. 5.

A. Translation Layer

We describe the role of our translation layer, which converts a quantized model produced by the chosen quantizer into the format we chose for QIGen. In particular, this includes a general strategy for storing multiple low-bit values into integers.

Data format. In the quantization formats we consider, each column of a weight matrix W is quantized nonuniformly (Section II), and each column uses the same partition and choice of bit-widths for the different groups. We store this information in three matrices: the quantized $m \times k$ weight matrix W , and the scale S and zero Z values for each column as depicted in Fig. 6. Each group in the partition has a corresponding size, bit, zero, and scale value assigned. Our translation layer converts any input quantization format into these structures. We implemented this conversion manually.

Storing Z and S for a quantized W requires $2Gk$ additional floating point values.

As mentioned already in Section II, leveraging specialized 8-bit integer operations instructions, such as `vpdpbusds` in AVX, is nontrivial. We pack consecutive low-bit weights in each column of W into a single integer. As the dot product is executed between bytes, and we work with integers with fewer bits, we permute the column such that there are 8-bit gaps between consecutive elements. Each group i is, therefore, a 2D array of integers with $(g_i b_i) / \ell$ rows and k columns, where

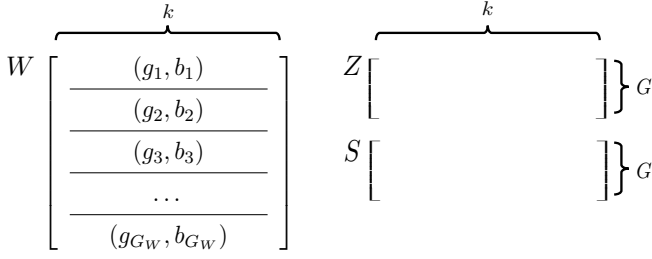


Fig. 6: After quantization, each weight matrix of the quantized model is stored as three matrices W , Z , and S that contain the quantized weights and the zero and scale values of each column, respectively.

we use g_i and b_i to represent the size and bit-width of the group, and ℓ to represent the size in bytes of the integer type we use for packing. For example, with 4-bit weights, we can pack 8 values into one 32-bit integer. Consider the following two columns, where we pack the 4-bit values it contains as:

$$\begin{pmatrix} 1 & 3 & 7 & 5 & 7 & 0 & 0 & 9 \\ 11 & 3 & 3 & 4 & 15 & 2 & 2 & 0 \end{pmatrix}^T \rightarrow \begin{pmatrix} 0 \times 17307059 \\ 0 \times \text{BF}323240 \end{pmatrix}^T$$

Note how the rows are permuted for efficient packing. While this process is straightforward for 4-bit weights, it becomes more complex when handling arbitrary bit-widths.

The total memory used to store a weight matrix of size $m \times k$ after quantization is thus the sum of the sizes of Z , S , and W :

$$M = 8Gk + \sum_{i=1}^G g_i b_i k / 8 \quad \text{bytes}, \quad (8)$$

and thus, the average number of bits per weight in W is

$$\rho = 8M/km. \quad (9)$$

Generalized packing for arbitrary bit-width. We now present the packing strategy that we use for any bit-width, including those that do not evenly divide the length of the integers used, such as 3-bit weights with 32-bit integers.

We consider general b -bit weights and 32-bit integers. Naturally, we can fit 32 consecutive weights into b integers. For example, we can fit 32 3-bit weights in 3 integers (96 bits), or we can fit 8 weights into 3 8-bit integers. Extracting the correct bits from an integer is then different for every value of b . For example, assume we quantize to $b = 3$ and pack the weights in 8-bit integers. We partition three consecutive 8-bit integers as

$$[3|3|2] \quad [1|3|3|1] \quad [2|3|3],$$

where each digit represents the number of bits, and each $|$ delimits separate values. In other words, to pack 8 3-bit values into 3 consecutive 8-bit integers, we can store two full 3-bit weights in the first integer, split the following weight by assigning 2 bits to the first byte and 1 bit to the second byte. Then, we store two 3-bit weights and split again. This partitioning strategy allows for an efficient storage solution for

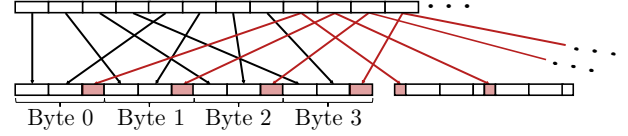


Fig. 7: Permutation of 3-bit weights in memory. We highlight the values that are split between different integer values.

```
void gevmm(float* A, uint32_t* B, float* C){
    for(int i = 0; i < n_groups; i++){
        quantize_input(A, buf, in_scale, in_reduction, i);
        for(int j = 0; j < K; j+=k_b){
            panel(buf, B, C, j, in_scale, in_reduction);
        }
    }
}
```

Fig. 8: High level GEVM implementation. First we quantize the input, then we execute all the vector-panel multiplications.

the weights, and can be easily applied to any combination of bits and integer sizes.

More specialized instructions, such as `vpdpbusds`, require the bits to be packed in a specific way. In particular, four consecutive weights need to be stored into four consecutive bytes, as explained above. We address this by applying an additional permutation on the stored matrix, each consecutive weight is stored with an offset of one byte. We visualize this permutation in Fig. 7 for the case of 3-bit values. At runtime, extracting groups of four consecutive values requires the use of fast logical operations (shifts, ands and ors).

Because of our packing strategy, we have a constraint on the sizes of the groups, meaning that for a group i , we need to have $g_i \bmod 32 = 0$. The quantizer `ExLlamaV2` that we use satisfies this condition as it only produces group sizes $\in \{32, 64, 128\}$.

B. Micro-Kernel Generation

Given a GEPM $C = AW$, we divide it into blocks, referred to as micro-kernels, $C_i = A_i W_i$ as shown in Fig. 2, where A_i is of size $n \times g_i$ and W_i is of size $g_i \times k_b$. We explain how we derive the GEPM blocking parameter k_b using a model in the following subsection.

The micro-kernels are split into two steps, an integer GEPM which accounts for most of the computation and a subsequent conversion to floating point values. Fig. 8 shows a single threaded skeleton of a C++ implementation for a GEVM, a particular instance of a GEPM. The first step quantizes the floating-point input into 8-bit values, which is done once per vector and can be parallelized. The second step performs the vector-matrix multiplication, which we parallelize over the columns to minimize contention for the output vector. The `panel` function shown is the one we generate, we will give more details in the following sections.

Integer GEPM. The integer GEPM phase has the goal of computing the product between arrays A and B from 8. To execute this product, we use `shift` and `and` instructions to extract the weights and then use `vpdpbusds` to execute the bulk of the computation. Supporting our generalized packing

strategy, the Z-Morton order layout and different kernel sizes lead to nontrivial generated code.

Floating-point conversion. The result of the integer GEPM is, in general, an integer matrix of size $n \times k_b$ which we convert to floating point. This conversion depends on the quantization format. If we consider the format in (4), the conversion requires an integer to floating-point conversion instruction (i.e. `vcvtdq2ps`) and an FMA with scale and zero point values.

Quantized micro-kernel. Given a quantized model, the first step is to extract all unique tuples (M, K, g_i, b_i) of group sizes and bit-widths. Given these two parameters we can then determine the exact form of the micro-kernels we need to generate. Estimating the number of registers needed at any point inside a micro-kernel is not obvious. This is because we need to decompress multiple rows of the weight matrix for every iteration of the innermost loop. Therefore, we execute a grid search over small powers of two to find the best-performant generation parameters. This grid search can take up to a few days depending on the amount of kernels to implement. The configuration can be stored afterwards for a specific CPU. At the end of this step, we have a map from each pair (M, K, g_i, b) to a micro-kernel implementation.

C. Matrix Packing

The BLIS-inspired GEPM implementation (see Section II and (20)) includes a step that copies the block W_b into a buffer. This ensures a unit stride access pattern over the entries of W_b and reduces TLB misses (20; 21). Since weight matrices are constant during inference, we remove this additional data movement at runtime by storing the matrix W in consecutive blocks at compile time. In our packing precomputation, we determine the parameter k_b needed for the GEPM. First, we require that the matrices W_b and C_b jointly fit in cache:

$$\frac{\varrho \cdot m k_b}{8} + 4n k_b < \gamma \implies k_b < \frac{\gamma}{\frac{\varrho \cdot m}{8} + 4n},$$

where γ is the size of the cache in bytes. Second, we require that k_b evenly splits k among the available cores. This means that we choose k_b as the largest power of 2 such that

$$k_b < \min\left(\frac{\gamma}{\frac{\varrho \cdot m}{8} + 4n}, \frac{k}{\text{\#of cores}}\right).$$

After we find k_b , we store each block of size $g_i \times k_b$ of the quantized matrix W consecutively following a blocked Z-curve order (22).

D. GEPM Generation

Finally, once all kernels and storage formats are constructed, we implement our GEPM functions for each layer of a quantized LLM as a sequence of function calls to the corresponding micro-kernels. To reduce code size, we merge function calls into fixed-size loops whenever consecutive groups share the same (g_i, b) configuration. Generating a sequence of specialized functions makes it easy to support arbitrary group sizes and bit-widths while maintaining high performance.

Additionally, at a macro level, since each matrix is stored such that blocks of size $m \times k_b$ are sequential in memory, we can easily parallelize the implementation by executing $C[:, j : j + k_b] += AW[:, j : j + k_b]$ on different threads.

Generating code for different ISAs. Manually targeting a different ISA requires rewriting all kernels from scratch. With QIGen, we only require manual intervention if the target architecture lacks hardware-specific instructions, such as `vpdpbusds`. The heavy lifting is instead done by LLVM. Since our kernels are mostly architecture agnostic—shifts, logical operations, FMAs and conversions are common instructions—our kernels generalize to different architectures.

We consider now the case of targeting ARM NEON, which supports a similar integer dot product instruction `vdotq_u32`. While `vpdpbusds` supports the multiplication of signed-unsigned 8-bit integers, `vdotq_u32` only supports the multiplication of unsigned-unsigned or signed-signed 8-bit integers. To integrate `vdotq_u32` in our program generator, we need to change the input format: before quantizing the input, we shift its values to have only non-negative numbers. We then execute our GEPM kernels and shift the final result. Mathematically, this transformation corresponds to

$$(x - \min(x) + \min(x))W = (x - \min(x))W + \min(x)W,$$

where $\min(x)$ is a matrix of size $n \times m$ containing the minimum values of each row of x , $x - \min(x)$ is the input we quantize using 8-bit unsigned integers, and $\min(x)(W)$ is the final shift. During matrix initialization, we precompute and store the column-wise sum of W , so that executing $\min(x)(W)$ corresponds to m multiplications. Applying and testing these changes took a single developer less than a day, showing the advantage of generation. The code is available on the GitHub repository.

E. Code Generation

QIGen generates the GEPM/GEVM kernels as specialized LLVM-IR and compiles them just-in-time using the LLVM ORC JIT APIs. In the following, we give implementation details on how we emit code for the micro-kernels and for the GEPM functions.

LLVM notation. We explain here the syntax of the LLVM instructions we use to generate code. We use an `IRBuilder` to emit LLVM IR. Most functions available on an `IRBuilder` are of the form `CreateFunc`, meaning it emits an LLVM IR instruction that computes `Func`. We have instructions that do loads, `getelementptr` (GEP), shifts (`Shl`, `LShr`) and logical instructions (`And`, `Or`). Additionally, LLVM provides a way to call specialized intrinsics, such as `vpdpbusd`, and user defined functions with `CreateCall`.

Micro-kernel generation. For a fixed target CPU and a fixed quantized model, QIGen enumerates all required kernel instances and generates one LLVM function per instance. Concretely, each generated micro-kernel implements a small GEPM/GEVM subproblem $C_i += A_i W_i$ where A_i and W_i are of size $[n \times g]$ and $[g \times k_b]$ respectively, with n batch size, g group size, and k_b panel width. We partially/fully unroll both

```

// B is an IRBuilder
switch(bits){
case 2:
...
case 3:
for(int ii = 0; ii < u_rows/4; ii+=8){
// loading input vector in a_i omitted
...
for(int jj = 0; jj < u_cols / simd_width; jj++){
b0_ptr = B.CreateGEP(...);
b1_ptr = B.CreateGEP(...);
b2_ptr = B.CreateGEP(...);
b0 = B.CreateAlignedLoad(..., b0_ptr);
b1 = B.CreateAlignedLoad(..., b1_ptr);
b2 = B.CreateAlignedLoad(..., b2_ptr);
b01_rs = B.CreateLShr(b0, 3);
b02_rs = B.CreateLShr(b0, 6);
b02_ls = B.CreateShl(b1, 2);
b10_rs = B.CreateLShr(b1, 1);
b11_rs = B.CreateLShr(b1, 4);
b12_rs = B.CreateLShr(b1, 7);
b12_ls = B.CreateShl(b2, 1);
b20_rs = B.CreateLShr(b2, 2);
b21_rs = B.CreateLShr(b2, 5);
b00 = B.CreateAnd(b0, 0x07070707);
b01 = B.CreateAnd(b01_rs, 0x07070707);
left0 = B.CreateAnd(b02_rs, 0x03030303);
right0 = B.CreateAnd(b02_ls, 0x04040404);
b02 = B.CreateOr(left0, right0);
b10 = B.CreateAnd(b10_rs, 0x07070707);
b11 = B.CreateAnd(b11_rs, 0x07070707);
left1 = B.CreateAnd(b12_rs, 0x01010101);
right1 = B.CreateAnd(b12_ls, 0x06060606);
b12 = B.CreateOr(left1, right1);
b20 = B.CreateAnd(b20_rs, 0x07070707);
b21 = B.CreateAnd(b21_rs, 0x07070707);
acc_ptr = B.CreateInBoundsGEP(...);
acc = B.CreateLoad(..., acc_ptr);
val = B.CreateCall(vpdpbusr, {acc, b00, a_i[0]});
val = B.CreateCall(vpdpbusr, {val, b01, a_i[1]});
val = B.CreateCall(vpdpbusr, {val, b02, a_i[2]});
val = B.CreateCall(vpdpbusr, {val, b10, a_i[3]});
val = B.CreateCall(vpdpbusr, {val, b11, a_i[4]});
val = B.CreateCall(vpdpbusr, {val, b12, a_i[5]});
val = B.CreateCall(vpdpbusr, {val, b20, a_i[6]});
val = B.CreateCall(vpdpbusr, {val, b21, a_i[7]});
B.CreateStore(val, acc_ptr);
}
}
break;
// rest of the code
}

```

Fig. 9: Generator code that emits the LLVM IR for a micro-kernel for $b = 3$. For clarity, we report only the code that unpacks the weights and computes the dot products, the code relating to addresses and types is omitted.

the loops over g and the loop over k_b depending on the selected register blocking. This enables LLVM to schedule shifts/ands, byte shuffles, dot products, and FMAs efficiently. We give in Fig. 9 the code that generates the main loop body for $b = 3$ for a vector-matrix kernel.

We use a switch case to handle each bit-width. The outermost loop (ii) iterates over rows of the micro-block, while the inner loop (jj), loads the values from the weight matrix, unpacks them by using shifts and masks. Finally, we generate the computation of the integer dot products using `vpdpbusd` (or `vdot` on ARM).

GEPM generation. For each layer of an LLM, we generate a function that computes a GEPM/GEVM function. Similarly to what is done for the micro-kernels, we emit directly LLVM

```

// B is an IRBuilder
for(int group_id = 0; group_id < n_groups; group_id++){
int g = groups[group_id];
int b = bits[group_id];
auto key = std::make_tuple(rows,cols,g,b);
// Code that computes the indices is omitted
...
auto Aptr = B.CreateInBoundsGEP(...);
auto Bptr = B.CreateInBoundsGEP(...);
auto Cptr = B.CreateInBoundsGEP(...);
B.CreateCall(functions[key], {A_ptr, B_ptr, C_ptr, ...});
}

```

Fig. 10: Generator code that emits the LLVM IR for a panel function. We report only the generation of the function calls. In this example we emit a function call for each group. In our code we merge multiple kernels that share the same quantization parameters.

IR. In particular, for all the quantization groups of a weight matrix, we generate a function call to the respective micro-kernel. We provide a snippet of code for the generation in Fig. 10. The generation is made up of one loop that iterates over the groups of the matrix. Then for each group we compute offset indices for input, output and weight matrices. Then we emit a function call with the correct addresses, to this end we use a map that links each set of parameters (M, K, g, b) to a micro-kernel.

V. EVALUATION

We now evaluate and analyze the performance of QIGen-generated code. Specifically, we compare the performance of our generated kernels against the manually-optimized kernels of `llama.cpp`, which provides uniform quantization only. First, we show that, for *uniform* quantization targeting, e.g., 4 bits per parameter, our generated kernels run faster than a comparable `llama.cpp` manual kernel. Then, we show our main result: in both a medium and a large LLM, we can achieve higher accuracy for the same overall kernel runtime, or, vice versa, we can significantly improve model execution and size for the same perplexity, especially for larger models.

Experimental setup. All tests are multi-threaded and are run on an Intel Xeon Silver 4410Y (Sapphire Rapids) 12-core CPU running at 2GHz and with 256GB of main memory. We disable both Turbo-Boost and hyper-threading. Measurements are conducted with cold cache following the approach of (23), to best reflect their use inside LLMs. In particular, we execute the benchmarks such that the weight matrices are not in cache (cold), while the input and output matrices are in cache (warm). QIGen is implemented in C++ and is compiled as a Python module using `pybind11`. We generate and compile LLVM-IR code using LLVM.

QIGen measurements are run using 8 threads. This is to reduce kernel size by avoiding the generation of cleanup code for matrix dimensions that are not evenly divisible. On the other hand, we report results for `llama.cpp` for two thread configurations: 8 and 12 threads. We do this for two reasons. First, using more threads increases the available bandwidth of the CPU, so we want to show a fair comparison against our

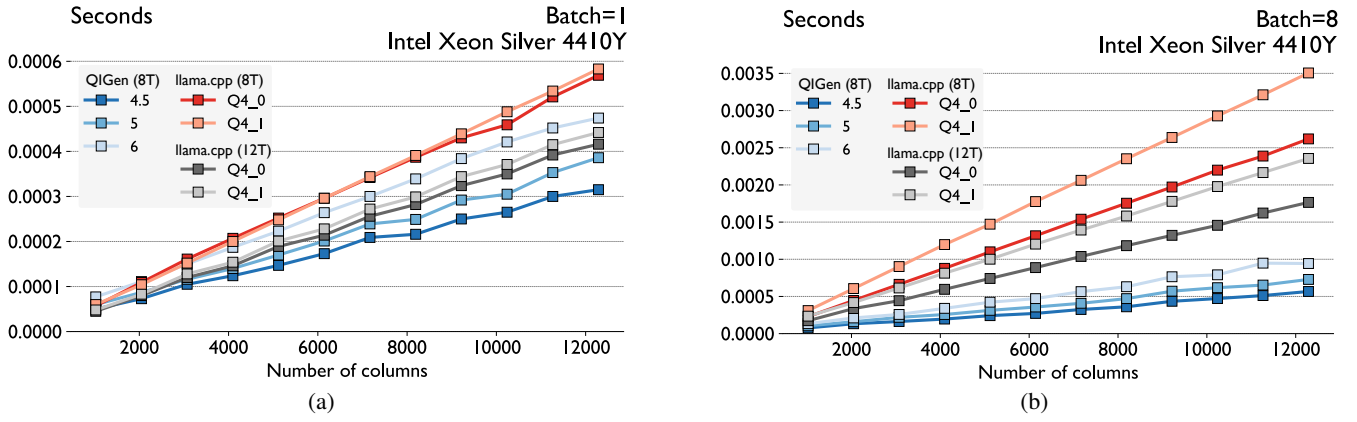


Fig. 11: Runtime comparison, for *uniform quantization*, between QIGen code and llama.cpp Q4_0 and Q4_1 formats. We keep the number of rows fixed to 4096 and we increase the number of columns. We consider batch size 1 (GEVM) in (a) and batch size 8 (GEPM) in (b). The plots have different scales on the y-axis.

generated code. Second, we show that even with the additional bandwidth provided by more threads, our kernels are just as fast, if not faster than llama.cpp.

We present performance results on the LLaMA3 family of open models, which are very broadly deployed at the time of writing. In particular, we consider the 8-billion (8B) parameter model, which is considered of medium size, and the larger and more capable 70-billion (70B) parameter model. We focus on measuring the execution time spent in the kernels, as this is the goal of our paper, and exclude orthogonal implementation details that arise when implementing an end-to-end inference pipeline. In the following plots, we denote our kernels by the bits per weight value of their models, computed via (9), and llama.cpp’s kernels by their formats.

Uniform quantization performance. In the first experiment, we consider the case of *uniform* quantization, which allows a direct comparison between QIGen-generated kernels and the manually optimized kernels provided by llama.cpp contributors. In particular, we compare against llama.cpp kernels uniformly quantized to approximately 4 bits, which come in two variants: Q4_0 and Q4_1, which implement symmetric and asymmetric quantization respectively. These two quantization schemes use half precision floating point for scales and zeros and use a group size of 32 elements. Therefore, they compress to 4.5 and 5 average bits per weight. For QIGen matrices, we quantize to 4 bits and vary group size: 32, 64, and 128 elements. Since we use single precision floating point for our scales and zeros, these correspond to 6, 5, and 4.5 bits per weight. We noticed in our experiments that matrices quantized to the same bits per weight tend to have similar runtimes as the main bottleneck is given by memory movement. The results are shown in Fig. 11 for batch sizes 1 (GEVM, left) and 8 (GEPM, right). For llama.cpp we show results for 8 threads and 12 in red and grey respectively.

We observe an overall speed-up in both cases, more pronounced for larger matrices and for batch size 8. For batch size 1 it is up to 1.3 times faster when comparing against

12 threads and 1.8 times faster when comparing against 8 threads, for the same bits per weight. For the GEPM case, we observe higher speedups: we are 3.4 times faster considering the 12 threads case and up to 5 times faster when considering 8 threads, again considering the same number of bits per weight. We believe that we observe this higher speedup for batch size 8 because the computation is no longer memory-bound and our generator is able to maintain high performance thanks to the tiling done at code generation time.

Layerwise measurements. In the second, and main experiment we perform a detailed measurement of QIGen-generated code for nonuniformly quantized LLMs, also considering the different layer matrices. The results, together with the fact that nonuniform quantization formats provide better perplexity at the same size, encapsulate our contribution.

Each layer of an LLM consists of seven matrices. The dimensions of these matrices are fixed for the entire model. We focus in the following on the bigger matrices, as they are responsible for a large percentage of the runtime. We consider nonuniformly quantized versions of the LLaMA3 8B and 70B models obtained via ExLlamaV2 by setting the target compression between 3 and 8 bits per weight. For llama.cpp we choose the Q4_0, Q4_1, Q5_0, Q5_1 and Q8_0, as they best resemble our current quantization format. In particular, Q4_0, Q5_0, and Q8_0 use a symmetric quantization scheme, while Q4_1 and Q5_1 use an asymmetric quantization scheme.

For each of the seven matrix types, we measure the runtime in seconds of our generated kernels and compare against the llama.cpp kernels. More specifically, for each matrix type, we measure the runtime of llama.cpp’s `ggml_mul_mat` operation on matrices of the respective dimensions, assuming independence of the matrix entries. We show in Fig. 12 only the runtime for the biggest matrices stored in the Up, Down and Gate projection layers. We report the measured total runtime for each matrix type in (a) the 8B, (b) the 70B models, and, as sum, the cumulative kernel time spent in the model. We estimate llama.cpp’s execution time by multiplying the

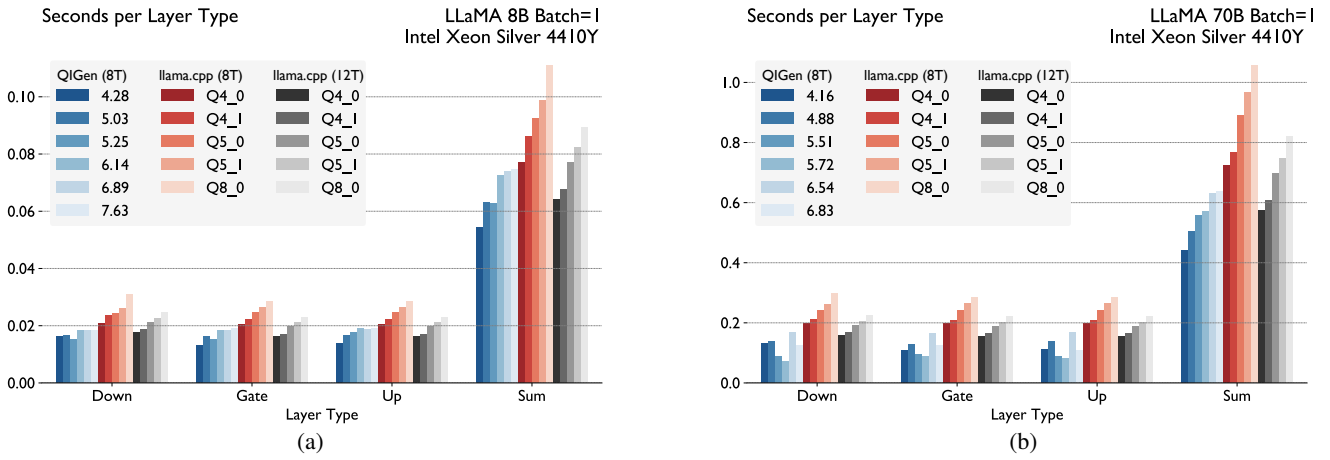


Fig. 12: Runtime measurement for kernels in various quantized LLaMA3 models for (a) 8B and (b) 70B parameters. We differentiate our models using their bits per weight computed as (9), and llama.cpp by their formats. Llama.cpp’s Q4_0, Q4_1, Q5_0, Q5_1 and Q8_0 formats correspond to 4.5, 5, 5.5, 6, 8.5 bits per weight respectively. We show the total runtime in seconds spent in the biggest layers (Down, Gate, Up), and the sum of seconds spent in all layers during inference. Lower is better. Each plot has a different scale on the y -axis.

measurement for one layer by the number of layers: 32 for the 8B model and 80 for the 70B model. For llama.cpp, we show again results for 8 and 12 threads. First, we observe that, as expected, the model runtimes decrease roughly monotonically in the bit-width. The few exceptions are likely due to non-uniform quantization, which may choose unusual bit widths like 3 or 5 that yield more complex instruction mixes. Second, when considering the same amount of compression in bits per weight, QIGen consistently performs better than the respective llama.cpp kernel, when considering 12 threads. When considering 8 threads for llama.cpp, QIGen is strictly better. Our largest model is faster than the most compressed (Q4_0) model.

Perplexity vs. runtime. We show in Fig. 13 the time in seconds it takes to execute all the matrix vector multiplications for the quantized models, and the corresponding perplexity. Lower is better on both axes. This result shows that using nonuniform quantization yields comparable accuracy for more performance. In particular, we achieve a speedup of $1.2\times$ and $1.29\times$ for the most accurate 8B and 70B models we consider.

Simulated TPOT and TTFT metrics. In addition to the layerwise results reported in Fig. 12, we also report simulated throughput (TPOT) and Time to First Token (TTFT). The former measures how many tokens per second are generated, and the latter measures how long it takes to generate the first token. In other words, TPOT measures the performance of the GEVM kernels while TTFT measures the runtime of a GEPM. We report the numbers for QIGen and llama.cpp in Table II. Again we use the bits per weight metric to indicate our quantized models and we use the formats’ names for llama.cpp. For TPOT higher values are better and for TTFT lower values are better. We use a batch size of 8 for the TTFT measurements. For llama.cpp we report the measurements for 8 threads and 12 threads as a pair.

TABLE II: Simulated throughput (TPOT, tokens/s) and time to first token (TTFT, s) for QIGen and llama.cpp on LLaMA3 8B and 70B models. QIGen models are identified by their bits per weight value, while llama.cpp models are identified by their quantization formats. TPOT measures GEVM performance (generation), and TTFT measures GEPM performance (prefill, batch size 8). Higher TPOT and lower TTFT are better. For llama.cpp we report results for both 8 and 12 threads as a pair, 8 threads on the left and 12 threads on the right.

Model	LLaMA 8B		LLaMA 70B		
	TPOT	TTFT	Model	TPOT	TTFT
<i>QIGen</i>					
4.28	18.44	0.16	4.16	2.27	1.37
5.03	15.87	0.18	4.88	1.98	1.51
5.25	15.95	0.20	5.51	1.80	1.60
6.14	13.77	0.18	5.72	1.76	1.52
6.89	13.54	0.21	6.54	1.59	1.53
7.63	13.39	0.17	6.83	1.57	1.53
<i>llama.cpp</i>					
Q4_0	(12.95, 15.62)	(0.36, 0.24)	Q4_0	(1.38, 1.74)	(3.48, 2.32)
Q4_1	(11.62, 14.82)	(0.49, 0.33)	Q4_1	(1.30, 1.65)	(4.69, 3.16)
Q5_0	(10.82, 12.99)	(0.48, 0.32)	Q5_0	(1.12, 1.44)	(4.71, 3.15)
Q5_1	(10.12, 12.17)	(0.60, 0.41)	Q5_1	(1.04, 1.34)	(5.80, 3.89)
Q8_0	(9.00, 11.21)	(0.42, 0.28)	Q8_0	(0.95, 1.22)	(4.07, 2.71)

VI. RELATED WORK

We review prior work on tools for computing on compressed data and related work on generating optimized linear kernels. **Compressed computing.** Working on compressed data, known as compressed domain processing, is a technique that has historical significance in fields that deal with large quantities of data. For instance, in image, video, and audio processing, there are attempts at operating on lossy compressed

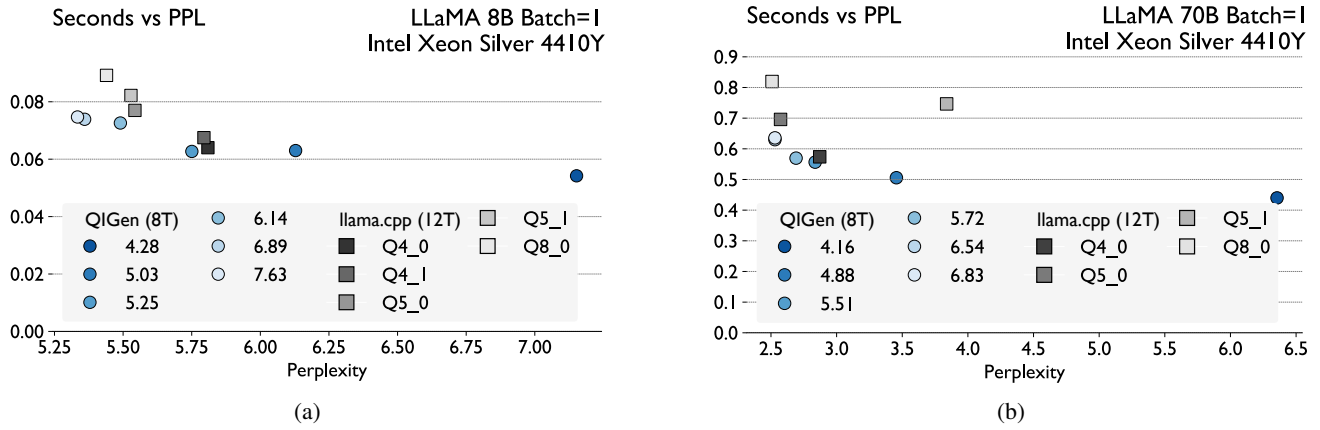


Fig. 13: Total seconds to run all the matrix-vector multiplications in a quantized model and their corresponding perplexity, for the dynamic format supported by QIGen (circles) vs. the equivalent llama.cpp uniform quantization (squares). Lower is better on both axes. QIGen provides a better trade-off across the entire Pareto frontier, for both model sizes.

(24; 25; 26) and lossless compressed (27) data.

Compression has been essential for performance in the domain of sparse linear algebra. The natural approach is not to store zeros as done, e.g., in the classical compressed sparse row format (CSR) (28). Additional ideas aim to further compress for repeating patterns or repeated indices (29; 30; 31; 32). Similar techniques have been proposed for sparse tensor operations (33; 34; 35; 36). For dense linear algebra, certain computations can first be done in low precision and then refined to full double precision (37; 38; 39).

Working with low precision has become the norm in fields such as machine learning. It is common now for GPUs and CPUs to support half-precision floating-point (FP16 or BF16). Moreover, low-bit integer support such as `int4` and 8/4-bit floating point values on NVIDIA GPUs and `AVX512_VNNI` instructions on Intel CPUs are specifically included to speed up machine learning workloads.

On the systems side, low precision matrix multiplication is implemented by various libraries (40; 41). However, most are limited to 8-bit and 4-bit kernels. Recent work has improved GPU inference using non-uniform quantization (42).

A related project, introducing compilation tools to LLMs, is MLC (14), based on a series of academic papers (43; 44; 45). However, the focus of the project is different as its aim is to compile whole models, and it does not support nonuniform quantization. However, we believe that our techniques could be integrated with MLC.

Here, we focus on system support for compression via nonuniform quantization, for a critical use case: inference on large language models on CPUs.

Program generation for performance. Computer generation of optimized kernels and programs has a long tradition in linear algebra and related domains, starting with PHiPAC (46) and ATLAS (47) for GEMM and other BLAS routines. (48) automatically maps higher-level functions to BLAS kernels. LGen (49) generates optimized code for small-scale computations. Generators for linear transforms include the FFTW codelet generator (50) and SPIRAL (51), which uses a domain-

specific language (DSL) for optimization. Halide (52) uses a DSL to target the image processing domain. More recently, DSLs have been explicitly developed to optimize machine learning (34; 53; 54; 43).

Runtime code generation has also been used dense linear algebra kernels (41), which JIT-generates high performance small-matrix multiplication kernels.

We see this paper as continuing this line of work by bringing the used techniques to the domain of LLM inference. In particular, we present a kernel generator for a particular form of linear algebra functions: vector-matrix and matrix-matrix multiplication in which the right operand is nonuniformly quantized, thus requiring packing and unpacking routines to perform the computation.

VII. CONCLUSIONS

Large language models (LLMs) are currently one of the hottest topics in information technology. Thus, there has been enormous developer effort to enable efficient LLM inference, producing carefully hand-coded and hand-optimized solutions. As described, once the models are quantized, the needed arithmetic becomes very complex and requires custom solutions not provided by prior linear algebra libraries.

We see the main contributions of this paper as two-fold. First, we bring the idea and techniques of automatic program generation to this field, reducing implementation effort and enabling rapid porting to new hardware. Second, we provide the first system support for nonuniform quantization, which requires a large number of custom kernels for different bit-widths. In turn, program generation allows nonuniform quantization to become practical.

Despite being a single-developer research effort, we have shown that QIGen can match or surpass the performance and accuracy of state-of-the-art manual solutions. As such, integrating QIGen in the custom inference pipelines developed by the large open source community has the potential to push the state-of-the-art in real-world LLM inference on CPUs.

REFERENCES

- [1] S. Black, S. Biderman, E. Hallahan, Q. Anthony, L. Gao, L. Golding, H. He, C. Leahy, K. McDonell, J. Phang, M. Pieler, U. S. Prashanth, S. Purohit, L. Reynolds, J. Tow, B. Wang, and S. Weinbach, "GPT-NeoX-20B: An Open-Source Autoregressive Language Model," in *Proceedings of BigScience Episode #5 – Workshop on Challenges & Perspectives in Creating Large Language Models*, A. Fan, S. Ilic, T. Wolf, and M. Gallé, Eds. virtual+Dublin: Association for Computational Linguistics, May 2022, pp. 95–136. doi: 10.18653/v1/2022.bigscience-1.9. [Online]. Available: <https://aclanthology.org/2022.bigscience-1.9/>
- [2] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. S. Koura, A. Sridhar, T. Wang, and L. Zettlemoyer, "OPT: Open Pre-trained Transformer Language Models," Jun. 2022. doi: 10.48550/arXiv.2205.01068.
- [3] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "LLaMA: Open and Efficient Foundation Language Models," Feb. 2023. doi: 10.48550/arXiv.2302.13971.
- [4] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, ser. NIPS '22. Red Hook, NY, USA: Curran Associates Inc., Nov. 2022, pp. 30318–30332.
- [5] T. Dettmers and L. Zettlemoyer, "The case for 4-bit precision: k-bit inference scaling laws," in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML'23, vol. 202. Honolulu, Hawaii, USA: JMLR.org, Jul. 2023, pp. 7750–7774.
- [6] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," Mar. 2023. doi: 10.48550/arXiv.2210.17323.
- [7] G. Park, B. Park, S. J. Kwon, B. Kim, Y. Lee, and D. Lee, "nuQmm: Quantized matmul for efficient inference of large-scale generative language models," 2022.
- [8] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: accurate and efficient post-training quantization for large language models," in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML'23, vol. 202. Honolulu, Hawaii, USA: JMLR.org, Jul. 2023, pp. 38087–38099.
- [9] Z. Yao, R. Y. Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, "ZeroQuant: Efficient and Affordable Post-Training Quantization for Large-Scale Transformers."
- [10] J. Lin, J. Tang, H. Tang, S. Yang, G. Xiao, and S. Han, "AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration," *GetMobile: Mobile Comp. and Comm.*, vol. 28, no. 4, pp. 12–17, Jan. 2025. doi: 10.1145/3714983.3714987. [Online]. Available: <https://dl.acm.org/doi/10.1145/3714983.3714987>
- [11] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*. Koblenz Germany: ACM, Oct. 2023, pp. 611–626. doi: 10.1145/3600006.3613165. [Online]. Available: <https://dl.acm.org/doi/10.1145/3600006.3613165>
- [12] NVIDIA, "Tensorrt-llm," 2024.
- [13] G. Gerganov, "llama.cpp," <https://github.com/ggerganov/llama.cpp>, 2023.
- [14] M. team, "MLC-LLM," 2023. [Online]. Available: <https://github.com/mlc-ai/mlc-llm>
- [15] A. Chavan, R. Magazine, S. Kushwaha, M. Debbah, and D. Gupta, "Faster and lighter LLMs: a survey on current challenges and way forward," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, ser. IJCAI '24, Jeju, Korea, Aug. 2024, pp. 7980–7988. doi: 10.24963/ijcai.2024/883. [Online]. Available: <https://doi.org/10.24963/ijcai.2024/883>
- [16] C. Lattner and V. Adve, "LLVM: a compilation framework for lifelong program analysis & transformation," in *International Symposium on Code Generation and Optimization, 2004. CGO 2004.*, Mar. 2004, pp. 75–86. doi: 10.1109/CGO.2004.1281665. [Online]. Available: <https://ieeexplore.ieee.org/document/1281665/>
- [17] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML'23, vol. 202. Honolulu, Hawaii, USA: JMLR.org, Jul. 2023, pp. 19274–19286.
- [18] M. Freitag and Y. Al-Onaizan, "Beam Search Strategies for Neural Machine Translation," in *Proceedings of the First Workshop on Neural Machine Translation*, T. Luong, A. Birch, G. Neubig, and A. Finch, Eds. Vancouver: Association for Computational Linguistics, Aug. 2017, pp. 56–60. doi: 10.18653/v1/W17-3207. [Online]. Available: <https://aclanthology.org/W17-3207/>
- [19] K. Goto and R. A. V. D. Geijn, "Anatomy of high-performance matrix multiplication," *ACM Trans. Math. Softw.*, vol. 34, no. 3, pp. 1–25, May 2008. doi: 10.1145/1356052.1356053. [Online]. Available: <https://dl.acm.org/doi/10.1145/1356052.1356053>
- [20] R. van de Geijn, *The Science of Programming Matrix Computations*, 01 2008.
- [21] K. Goto and R. van de Geijn, "On reducing tlb misses in matrix multiplication," 2002.
- [22] V. Valsalam and A. Skjellum, "A framework for high-performance matrix multiplication based on hierarchical abstractions, algorithms and optimized low-level kernels," *Concurrency and Computation: Practice and Experience*, vol. 14, no. 10, pp. 805–839, Aug. 2002. doi: 10.1002/cpe.630., publisher:

- John Wiley & Sons, Ltd. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/cpe.630>
- [23] G. Ofenbeck, R. Steinmann, V. Caparros, D. G. Spampinato, and M. Püschel, "Applying the roofline model," in *2014 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, Mar. 2014, pp. 76–85. doi: 10.1109/ISPASS.2014.6844463. [Online]. Available: <https://ieeexplore.ieee.org/document/6844463/>
 - [24] F. Arman, A. Hsu, and M.-Y. Chiu, "Image processing on compressed data for large video databases," in *Proceedings of the first ACM international conference on Multimedia - MULTIMEDIA '93*. Anaheim, California, United States: ACM Press, 1993, pp. 267–272. doi: 10.1145/166266.166297. [Online]. Available: <http://portal.acm.org/citation.cfm?doid=166266.166297>
 - [25] M. A. Broadhead and C. B. Owen, "Direct manipulation of MPEG compressed digital audio," in *Proceedings of the third ACM international conference on Multimedia - MULTIMEDIA '95*. San Francisco, California, United States: ACM Press, 1995, pp. 499–507. doi: 10.1145/217279.215314. [Online]. Available: <http://portal.acm.org/citation.cfm?doid=217279.215314>
 - [26] S.-F. Chang and D. Messerschmitt, "Manipulation and compositing of MC-DCT compressed video," *IEEE Journal on Selected Areas in Communications*, vol. 13, no. 1, pp. 1–11, Jan. 1995. doi: 10.1109/49.363151. [Online]. Available: <https://ieeexplore.ieee.org/document/363151/>
 - [27] W. Thies, S. Hall, and S. Amarasinghe, "Manipulating lossless video in the compressed domain," in *Proceedings of the 17th ACM international conference on Multimedia*. Beijing China: ACM, Oct. 2009, pp. 331–340. doi: 10.1145/1631272.1631319. [Online]. Available: <https://dl.acm.org/doi/10.1145/1631272.1631319>
 - [28] E.-J. Im, K. Yelick, and R. Vuduc, "Sparsity: Optimization Framework for Sparse Matrix Kernels," *The International Journal of High Performance Computing Applications*, vol. 18, no. 1, pp. 135–158, Feb. 2004. doi: 10.1177/1094342004041296. [Online]. Available: <https://journals.sagepub.com/doi/10.1177/1094342004041296>
 - [29] R. Vuduc, J. Demmel, K. Yelick, S. Kamil, R. Nishtala, and B. Lee, "Performance Optimizations and Bounds for Sparse Matrix-Vector Multiply," in *SC '02: Proceedings of the 2002 ACM/IEEE Conference on Supercomputing*, Nov. 2002, pp. 26–26. doi: 10.1109/SC.2002.10025. iISSN: 1063-9535. [Online]. Available: <https://ieeexplore.ieee.org/document/1592862/>
 - [30] M. Belgin, G. Back, and C. J. Ribbens, "Pattern-based sparse matrix representation for memory-efficient SMVM kernels," in *Proceedings of the 23rd international conference on Supercomputing*. Yorktown Heights NY USA: ACM, Jun. 2009, pp. 100–109. doi: 10.1145/1542275.1542294. [Online]. Available: <https://dl.acm.org/doi/10.1145/1542275.1542294>
 - [31] J. Willcock and A. Lumsdaine, "Accelerating sparse matrix computations via data compression," in *Proceedings of the 20th annual international conference on Supercomputing*. Cairns Queensland Australia: ACM, Jun. 2006, pp. 307–316. doi: 10.1145/1183401.1183444. [Online]. Available: <https://dl.acm.org/doi/10.1145/1183401.1183444>
 - [32] K. Kourtis, G. Goumas, and N. Koziris, "Improving the Performance of Multithreaded Sparse Matrix-Vector Multiplication Using Index and Value Compression," in *2008 37th International Conference on Parallel Processing*, Sep. 2008, pp. 511–519. doi: 10.1109/ICPP.2008.62. iISSN: 2332-5690. [Online]. Available: <https://ieeexplore.ieee.org/document/4625888/>
 - [33] H. Xu, T. B. Schardl, M. Pellauer, and J. S. Emer, "Optimizing Compression Schemes for Parallel Sparse Tensor Algebra," in *2023 Data Compression Conference (DCC)*, Mar. 2023, pp. 373–373. doi: 10.1109/DCC55655.2023.00084. iISSN: 2375-0359. [Online]. Available: <https://ieeexplore.ieee.org/document/10125541/>
 - [34] F. Kjolstad, S. Chou, D. Lugato, S. Kamil, and S. Amarasinghe, "Taco: A tool to generate tensor algebra kernels," in *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Oct. 2017, pp. 943–948. doi: 10.1109/ASE.2017.8115709. [Online]. Available: <https://ieeexplore.ieee.org/document/8115709/>
 - [35] D. Donenfeld, S. Chou, and S. Amarasinghe, "Unified Compilation for Lossless Compression and Sparse Computing," in *2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, Apr. 2022, pp. 205–216. doi: 10.1109/CGO53902.2022.9741282. [Online]. Available: <https://ieeexplore.ieee.org/document/9741282/>
 - [36] R. Patel, W. Ahrens, and S. Amarasinghe, "SySTeC: A Symmetric Sparse Tensor Compiler," in *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization*. Las Vegas NV USA: ACM, Mar. 2025, pp. 47–62. doi: 10.1145/3696443.3708919. [Online]. Available: <https://dl.acm.org/doi/10.1145/3696443.3708919>
 - [37] J. Langou, J. Langou, P. Luszczek, J. Kurzak, A. Buttari, and J. Dongarra, "Exploiting the Performance of 32 bit Floating Point Arithmetic in Obtaining 64 bit Accuracy (Revisiting Iterative Refinement for Linear Systems)," in *SC '06: Proceedings of the 2006 ACM/IEEE Conference on Supercomputing*, Nov. 2006, pp. 50–50. doi: 10.1109/SC.2006.30. [Online]. Available: <https://ieeexplore.ieee.org/document/4090224/>
 - [38] A. Buttari, J. Dongarra, J. Langou, J. Langou, P. Luszczek, and J. Kurzak, "Mixed Precision Iterative Refinement Techniques for the Solution of Dense Linear Systems," *The International Journal of High Performance Computing Applications*, vol. 21, no. 4, pp. 457–466, Nov. 2007. doi: 10.1177/1094342007084026.
 - [39] A. Haidar, S. Tomov, J. Dongarra, and N. J. Higham, "Harnessing GPU Tensor Cores for Fast

- FP16 Arithmetic to Speed up Mixed-Precision Iterative Refinement Solvers,” in *SC18: International Conference for High Performance Computing, Networking, Storage and Analysis*, Nov. 2018, pp. 603–613. doi: 10.1109/SC.2018.00050. [Online]. Available: <https://ieeexplore.ieee.org/document/8665777/>
- [40] D. Khudia, J. Huang, P. Basu, S. Deng, H. Liu, J. Park, and M. Smelyanskiy, “FBGEMM: Enabling High-Performance Low-Precision Deep Learning Inference,” Jan. 2021. doi: 10.48550/arXiv.2101.05615.
- [41] A. Heinecke, G. Henry, M. Hutchinson, and H. Pabst, “LIBXSMM: Accelerating Small Matrix Multiplications by Runtime Code Generation,” in *SC ’16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, Nov. 2016, pp. 981–991. doi: 10.1109/SC.2016.83., iSSN: 2167-4337. [Online]. Available: <https://ieeexplore.ieee.org/document/7877162/>
- [42] P. Zhao and X. Yuan, “GANQ: GPU-Adaptive Non-Uniform Quantization for Large Language Models,” Jun. 2025. doi: 10.48550/arXiv.2501.12956.
- [43] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, M. Cowan, H. Shen, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, “TVM: an automated end-to-end optimizing compiler for deep learning,” in *Proceedings of the 13th USENIX conference on Operating Systems Design and Implementation*, ser. OSDI’18. USA: USENIX Association, Oct. 2018, pp. 579–594.
- [44] S. Feng, B. Hou, H. Jin, W. Lin, J. Shao, R. Lai, Z. Ye, L. Zheng, C. H. Yu, Y. Yu, and T. Chen, “TensorIR: An Abstraction for Automatic Tensorized Program Optimization,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. Vancouver BC Canada: ACM, Jan. 2023, pp. 804–817. doi: 10.1145/3575693.3576933. [Online]. Available: <https://dl.acm.org/doi/10.1145/3575693.3576933>
- [45] J. Shao, X. Zhou, S. Feng, B. Hou, R. Lai, H. Jin, W. Lin, M. Masuda, C. H. Yu, and T. Chen, “Tensor program optimization with probabilistic programs,” in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, ser. NIPS ’22. Red Hook, NY, USA: Curran Associates Inc., Nov. 2022.
- [46] J. Bilmes, K. Asanovic, C.-W. Chin, and J. Demmel, “Optimizing matrix multiply using PHiPAC: a portable, high-performance, ANSI C coding methodology,” in *Proceedings of the 11th international conference on Supercomputing - ICS ’97*. Vienna, Austria: ACM Press, 1997, pp. 340–347. doi: 10.1145/263580.263662. [Online]. Available: <http://portal.acm.org/citation.cfm?doid=263580.263662>
- [47] R. Clint Whaley, A. Petitet, and J. J. Dongarra, “Automated empirical optimizations of software and the atlas project,” *Parallel Computing*, vol. 27, Jan. 2001. doi: 10.1016/S0167-8191(00)00087-9.
- tailored linear algebra algorithms: A search-based approach,” *The International Journal of High Performance Computing Applications*, vol. 27, no. 4, pp. 426–439, Nov. 2013. doi: 10.1177/1094342013494428., publisher: SAGE Publications Ltd STM. [Online]. Available: <https://doi.org/10.1177/1094342013494428>
- [49] D. G. Spampinato and M. Püschel, “A Basic Linear Algebra Compiler,” in *Proceedings of Annual IEEE/ACM International Symposium on Code Generation and Optimization*. Orlando FL USA: ACM, Feb. 2014, pp. 23–32. doi: 10.1145/2581122.2544155. [Online]. Available: <https://dl.acm.org/doi/10.1145/2581122.2544155>
- [50] M. Frigo, “A fast Fourier transform compiler,” *SIGPLAN Not.*, vol. 34, no. 5, pp. 169–180, May 1999. doi: 10.1145/301631.301661. [Online]. Available: <https://dl.acm.org/doi/10.1145/301631.301661>
- [51] M. Puschel, J. Moura, J. Johnson, D. Padua, M. Veloso, B. Singer, J. Xiong, F. Franchetti, A. Gacic, Y. Voronenko, K. Chen, R. Johnson, and N. Rizzolo, “SPIRAL: Code Generation for DSP Transforms,” *Proceedings of the IEEE*, vol. 93, no. 2, pp. 232–275, Feb. 2005. doi: 10.1109/JPROC.2004.840306. [Online]. Available: <https://ieeexplore.ieee.org/document/1386651/>
- [52] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, “Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines,” in *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation*. Seattle Washington USA: ACM, Jun. 2013, pp. 519–530. doi: 10.1145/2491956.2462176. [Online]. Available: <https://dl.acm.org/doi/10.1145/2491956.2462176>
- [53] A. Bik, P. Koanantakool, T. Shpeisman, N. Vasilache, B. Zheng, and F. Kjolstad, “Compiler Support for Sparse Tensor Computations in MLIR,” *ACM Trans. Archit. Code Optim.*, vol. 19, no. 4, pp. 1–25, Dec. 2022. doi: 10.1145/3544559. [Online]. Available: <https://dl.acm.org/doi/10.1145/3544559>
- [54] M. Bansal, O. Hsu, K. Olukotun, and F. Kjolstad, “Mosaic: An Interoperable Compiler for Tensor Algebra,” *Proc. ACM Program. Lang.*, vol. 7, no. PLDI, pp. 394–419, Jun. 2023. doi: 10.1145/3591236. [Online]. Available: <https://dl.acm.org/doi/10.1145/3591236>
- [48] D. Fabregat-Traver and P. Bientinesi, “Application-